Рекуррентные нейронные сети (часть 1)

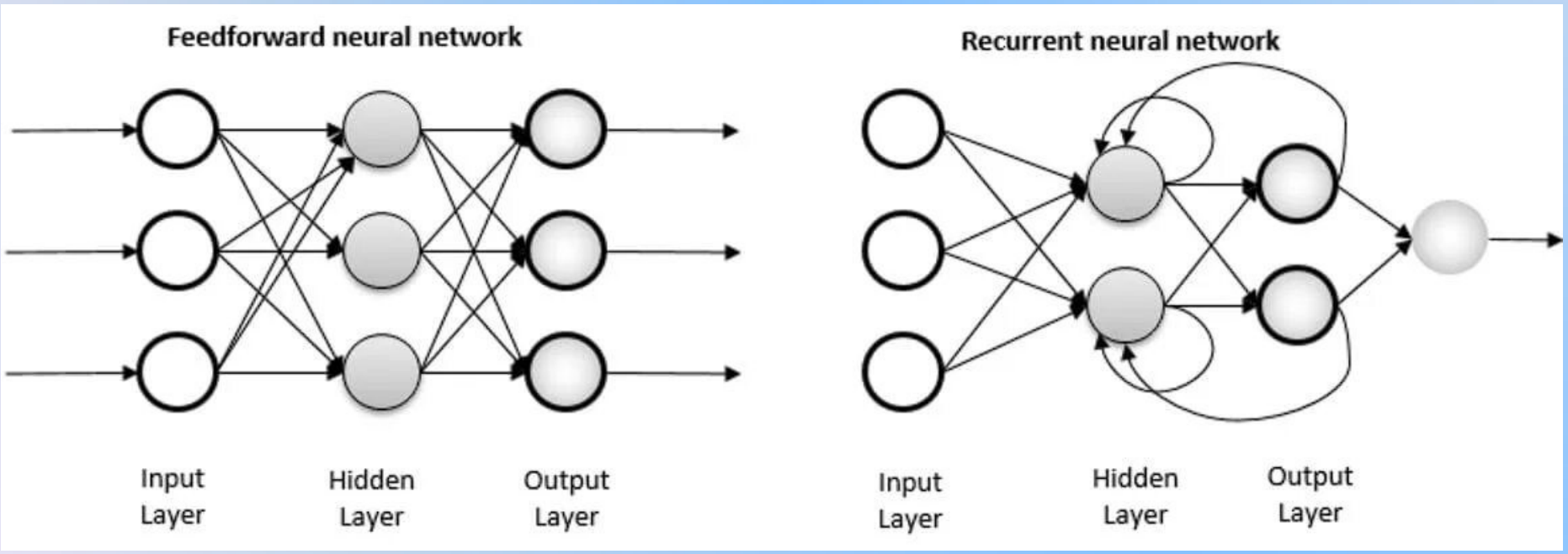
Рекуррентные нейронные сети (часть 1) Лекция 10

Седых И.А.

Рекуррентная нейронная сеть (RNN)

Рекуррентная нейронная сеть (RNN) - нейросеть, в которой есть рекуррентные связи, то есть выход с предыдущего шага подается на вход текущему. Это делает RNN универсальным аппроксиматором для последовательностей. Более того, рекуррентная сеть обладает полнотой по Тьюрингу, то есть при определенных условиях может моделировать произвольные вычисления.

Идея: в сеть прямого распространения добавить рекуррентные связи.



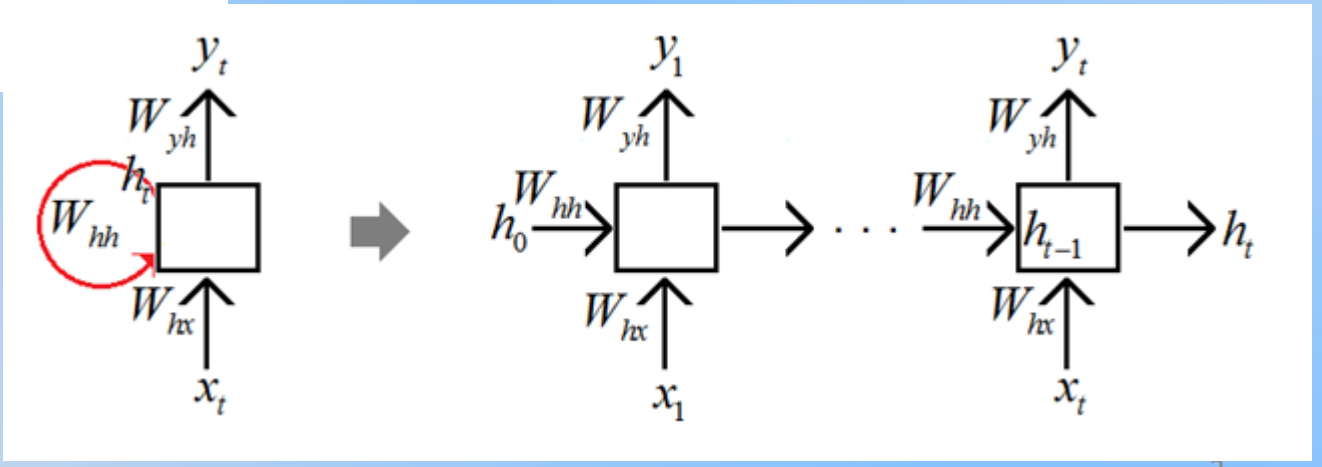
Принцип работы рекуррентной нейронной сети

Рекуррентные нейросети уникальны тем, что на каждом шаге обработки они учитывают не только текущий вход, но и предыдущее состояние сети.

Два ключевых элемента в работе RNN:

1. **Выход сети** - результат обработки текущего шага.
2. **Скрытое состояние** - внутренняя память сети о предыдущих шагах.

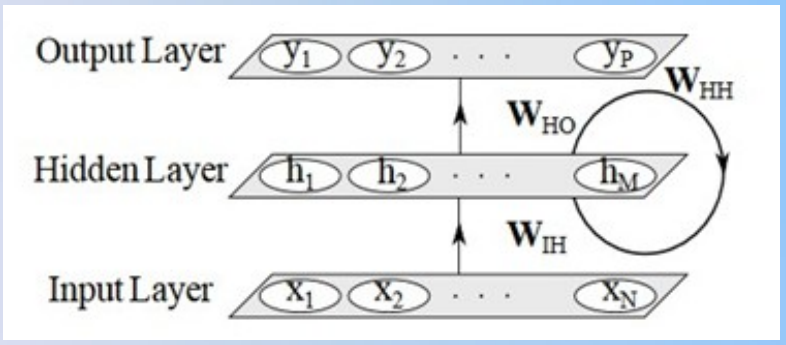
- На каждом шаге сеть получает:
  - Текущий вход x_t
  - Предыдущее скрытое состояние h_{t-1}
- Формирует:
  - Новое скрытое состояние h_t
  - Выходной сигнал y_t

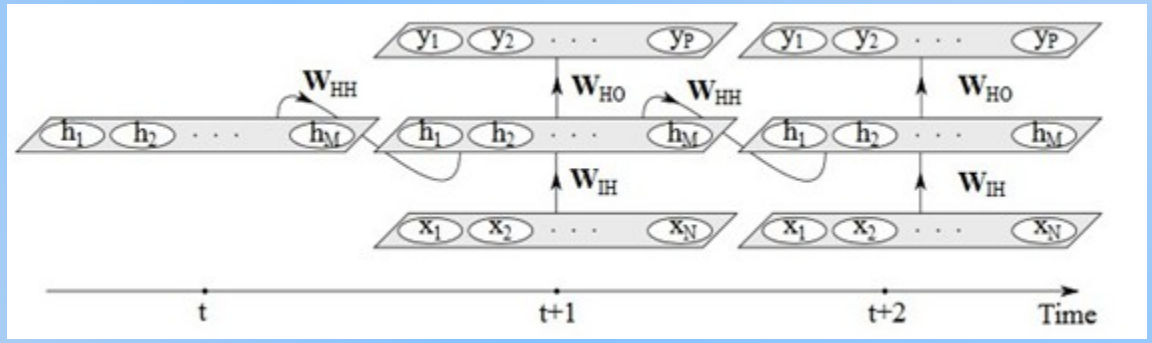


Рекуррентная нейросеть (RNN) – основная идея

Разделение параметров – основная идея RNN, такая же, как в свертках. Матрицы весов одинаковы на каждом шаге. Это позволяет обучить одну модель, а затем применять её к последовательностям любой длины.

На рисунке слева – свёрнутая RNN, справа – развёрнутая RNN во времени.





Преимущества и применение рекуррентных сетей

Ключевые достоинства RNN:

- **сохранение контекста** - сеть «помнит» предыдущие входы;
- **масштабируемость** - легко работает с последовательностями разной длины;
- **эффективность** - параметры разделяются между шагами.

Примеры использования RNN :

- обработка текста - анализ контекста предложений;
- распознавание речи - учёт последовательности звуков;
- анализ временных рядов - работа с последовательностями данных и др.

Базовый блок простых рекуррентных сетей (Vanilla RNN)

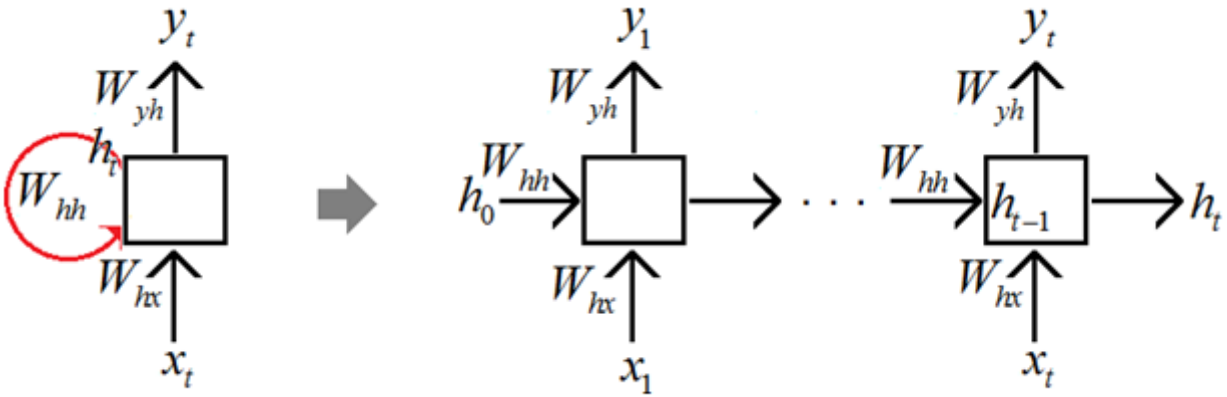
Рассмотрим однослойную сеть. Инициализируется h_0. Затем на каждом шаге t новое скрытое состояние h_t вычисляется как нелинейное преобразование (обычно tanh или ReLU) от линейной комбинации предыдущего состояния h_t-1 и текущего входа x_t. Далее выход y_t - это линейный слой от h_t плюс, возможно, нелинейность (например, softmax для классификации).

Базовые уравнения:

$$h_0 = \text{init}()$$
$$h_1 = \sigma(W_{hh}h_0 + W_{hx}x_1 + b_h)$$
$$\dots$$
$$h_t = \sigma(W_{hh}h_{t-1} + W_{hx}x_t + b_h)$$
$$y_t = g(W_{yh}h_t + b_y)$$

где:

- h_t — текущее скрытое состояние
- h_{t-1} — предыдущее состояние
- x_t — текущий вход
- W — весовые матрицы
- b_h — вектор смещения



Компактная форма записи

У нас есть три группы параметров, и все они **общие для всех временных шагов**:

1. W_{hh} — матрица весов для рекуррентной связи (от предыдущего скрытого состояния к текущему).
2. W_{hx} — матрица весов для входной связи (от текущего входа к скрытому состоянию).
3. W_{yh} — матрица весов для выходной связи (от скрытого состояния к выходу).

И два вектора смещений (bias): b_h и b_y.

Компактная форма записи для однослойной RNN:

$$h_t = \sigma(W_{hh}h_{t-1} + W_{hx}x_t + b_h) \quad \sim \quad h_t = \sigma\left([W_{hh}, W_{hx}]\begin{bmatrix} h_{t-1} \\ x_t \end{bmatrix} + b_h\right) \equiv \sigma\left(W[h_{t-1}; x_t] + b_h\right)$$

- Квадратные скобки $[h_{t-1}; x_t]$ обозначают **вертикальную конкатенацию** векторов (один под другим).
- W — это **горизонтальная конкатенация** матриц W_{hh} и W_{hx}.
- Итоговая размерность: если $h_{t-1} \in \mathbb{R}^{d_h}$, $x_t \in \mathbb{R}^{d_x}$, то $[h_{t-1}; x_t] \in \mathbb{R}^{d_h+d_x}$, а $W \in \mathbb{R}^{d_h \times (d_h+d_x)}$.

Два вида базовых архитектур рекуррентных сетей - 1

1. Сеть Джордана (1986) - первая обучаемая рекуррентная сеть. Майкл Джордан первым осознал, что обратное распространение можно применить к сетям с обратными связями. Его архитектура использовала **контекстные блоки** (context units), которые сохраняли предыдущие выходы сети и подавали их обратно на скрытый слой.

Сеть Джордана состоит из четырёх типов нейронов:

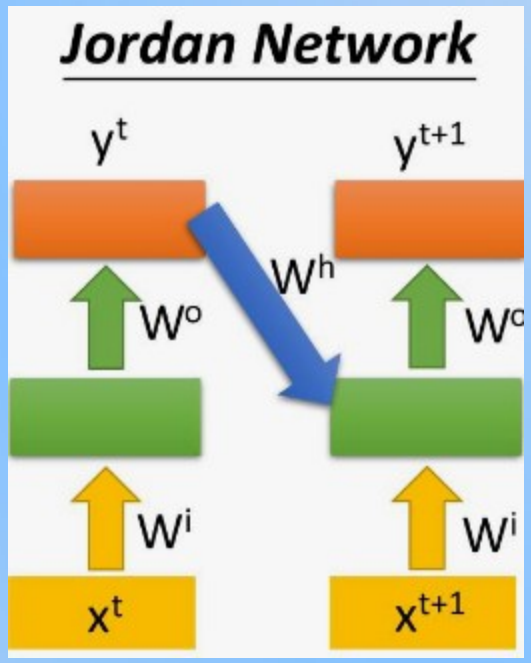
a. **Входные нейроны** (input units) получают внешний сигнал.
b. **Скрытые нейроны** (hidden units) основной вычислительный слой.
c. **Выходные нейроны** (output units) выдают результат.
d. **Контекстные нейроны** (context units) хранят копию выходов с предыдущего шага.

Уравнения сети Джордана:

$$h_t = \sigma_h(W_h x_t + U_h y_{t-1} + b_h)$$

$$y_t = \sigma_y(W_y h_t + b_y)$$

Рекуррентная связь идёт от **выходного** слоя к **скрытому** через **контекстные блоки**. Контекстные блоки хранят y_{t-1} и подают его на вход скрытому слою вместе с x_t.



Два вида базовых архитектур рекуррентных сетей - 2

2. Сеть Элмана (1990) - Simple Recurrent Network (SRN). Джеффри Элман предложил модификацию, которая оказалась более эффективной на практике - рекуррентная связь от **скрытого** слоя к самому себе. Эта архитектура стала **классической** реализацией простой RNN.

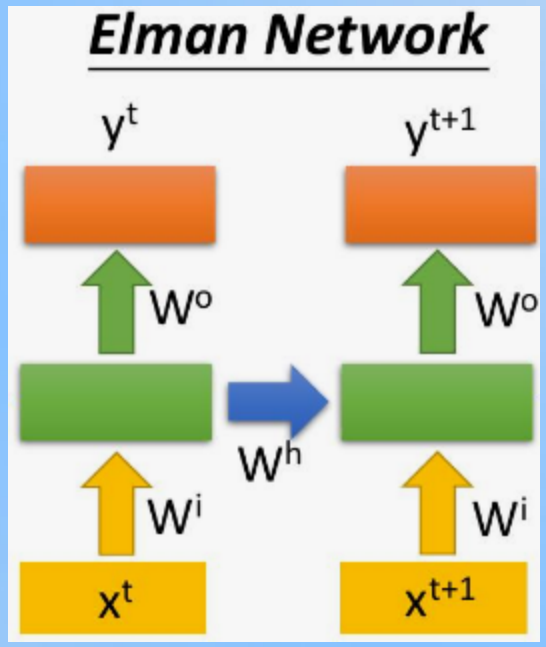
Сеть Элмана состоит из **трёх типов нейронов**: входной, скрытый (рекуррентный), выходной. Контекстные блоки получают копию скрытого состояния h_t и подают её обратно на скрытый слой на следующем шаге. Веса связей от скрытого слоя к контекстным блокам фиксированы и равны 1 (это не обучаемые параметры, а просто копирование).

Уравнения сети Элмана:

$$h_t = \sigma_h(W_h x_t + U_h h_{t-1} + b_h)$$

$$y_t = \sigma_y(W_y h_t + b_y)$$

Сеть Элмана более гибкая, поскольку размерность скрытого состояния можно выбирать независимо от размерности выхода.



Потенциальные проблемы рекуррентных сетей - 1

1. Забывание (Forgetting).

Предположим, мы обрабатываем длинное предложение из 100 слов. На шаге 1 мы прочитали слово «Я». На шаге 50 - слово «живу». На шаге 100 - слово «Москве».

Смысл всего предложения зависит от того, что «Я» - это подлежащее, а «Москве» - это место. Но сможет ли RNN на шаге 100 «помнить», что в начале было «Я»?

В нашей формуле: $h_{100} = \tanh(W \cdot [h_{99}; x_{100}] + b)$

Состояние h_{99} уже содержит информацию о прошлом, но в **сжатом**, **искажённом** виде. Каждый раз при применении tanh и умножении на W информация теряется. Это похоже на игру в «испорченный телефон»: после 100 передач исходное сообщение почти неразличимо.

Почему это происходит?

Матрица W - одна и та же на каждом шаге. Если её собственные значения меньше 1 по модулю, то при многократном умножении сигнал экспоненциально затухает. Компактная форма записи делает этот эффект очевидным: у нас есть **степень матрицы** W^t, которая действует на начальное состояние.

Если представить, что каждый шаг - это фильтр, который пропускает только 70% информации. После 10 шагов останется $0.7^{10} \approx 0.028$ - меньше 3%. После 100 шагов - практически ноль. Это и есть **забывание**.

Потенциальные проблемы рекуррентных сетей - 2

2. Градиенты.

Вторая проблема тесно связана с первой, но проявляется при **обучении**, а не при прямом распространении.

Когда мы обучаем RNN методом обратного распространения во времени, ошибка с последнего шага должна «дойти» до первого. Формула градиента содержит произведение якобианов:

$$\frac{\partial h_t}{\partial h_k} = \prod_{i=k+1}^{t} \frac{\partial h_i}{\partial h_{i-1}} = \prod_{i=k+1}^{t} W^T \cdot \mathrm{diag}(\sigma'(\dots))$$

В компактной форме записи это произведение выглядит как **степень матрицы** W^T с дополнительными диагональными множителями от производной нелинейности.

Следовательно:

1. Если собственные значения *W* больше 1, то градиенты **взрываются** (становятся огромными, обновления весов уходят в бесконечность, loss = NaN).
2. Если собственные значения меньше 1, то градиенты **затухают** (становятся практически нулевыми, ранние шаги не получают сигнала об ошибке).

В обычной глубокой сети на каждом слое **своя матрица**, и их произведение не обязательно приводит к экспоненциальному росту или затуханию. А в рекуррентной сети **одна и та же матрица возводится в степень**.

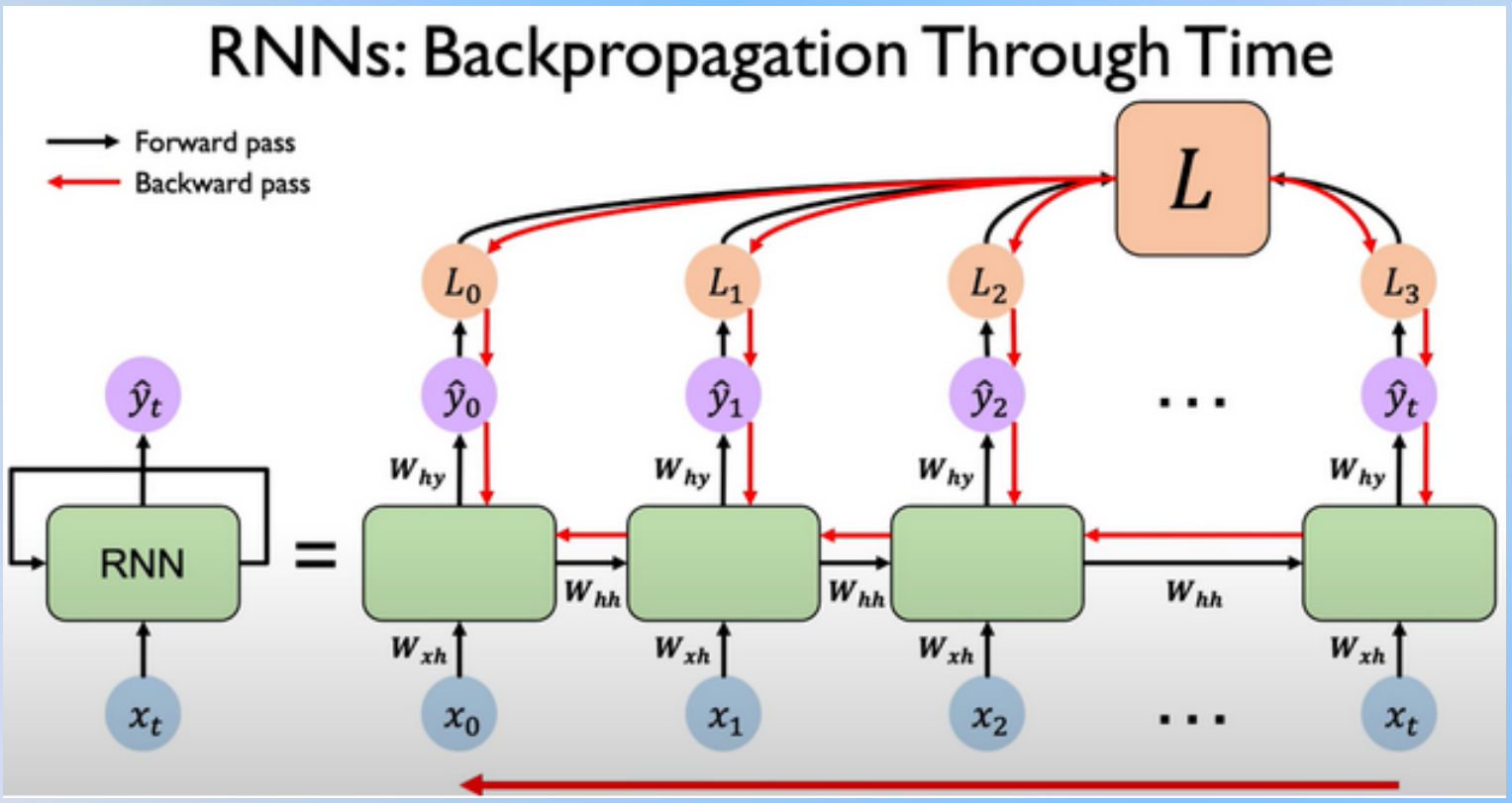
Это **ключевое отличие** RNN от глубоких сетей прямого распространения.

Решение проблем простых рекуррентных сетей

1. **Простая RNN не подходит для длинных последовательностей.** Если нужно обработать текст в 500 слов или временной ряд в 1000 точек, простая RNN не справится. Она либо забудет начало, либо градиенты взорвутся.
2. **Нужны специальные архитектуры.** LSTM и GRU - это попытки решить проблему затухающих градиентов, добавляя «пути обхода» для градиента (например, ячейку памяти C_t, которая обновляется линейно).
3. **Нужны специальные приёмы обучения.** Clipping градиентов (обрезка) помогает со взрыванием. Ортогональная инициализация W помогает с затуханием. Teacher forcing помогает с накоплением ошибки.
4. **Короткие последовательности.** Для последовательностей длиной 10-20 шагов простая RNN работает вполне нормально.

Обучение RNN методом BPTT

Обучение RNN происходит с помощью **алгоритма обратного распространения во времени** (Backpropagation Through Time, BPTT). Мы как бы «разворачиваем» сеть в глубокую прямую сеть, где число слоёв равно длине последовательности. Проходим по этой развернутой сети вперед, вычисляем ошибку, а затем обратно, корректируя веса. При этом веса на всех развернутых шагах остаются общими, их градиенты суммируются.



Авторы BPTT

1. Пол Вербос (Paul Werbos) считается общепризнанным автором BPTT, который формально описал и предложил этот метод в своей работе 1990 года «Backpropagation through time: what it does and how to do it» («Обратное распространение ошибки во времени: что оно делает и как его применять»).

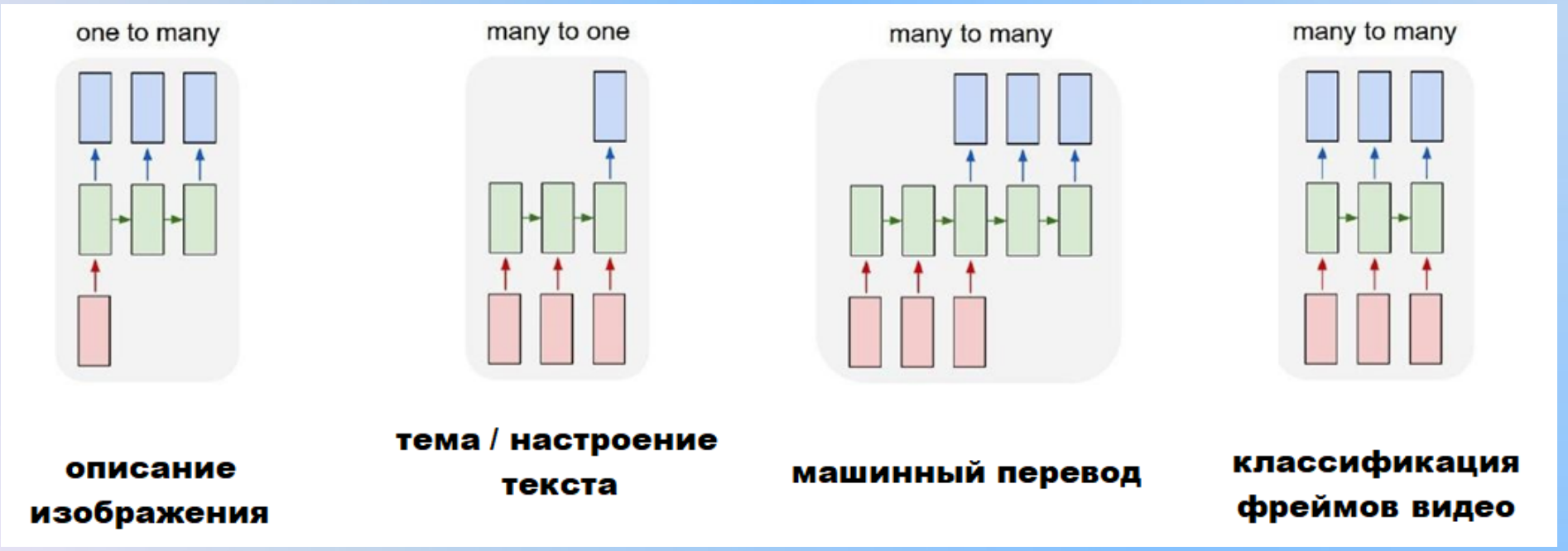
2. Рональд Дж. Вильямс и Дэвид Ципсер (Ronald J. Williams and David Zipser) в 1989 году предложили и применили алгоритм для обучения рекуррентных сетей в реальном времени (Real-Time Recurrent Learning, RTRL), а в 1992 году адаптировали эти идеи для BPTT. Их работы заложили основу для применения градиентных методов в RNN.

3. Зепп Хохрайтер и Юрген Шмидхубер (Sepp Hochreiter and Jürgen Schmidhuber) хотя не являются непосредственными изобретателями BPTT, именно их анализ проблем этого алгоритма (в частности, исчезающего градиента) привел к созданию LSTM в 1997 году. После появления LSTM, BPTT стал стандартным методом и для ее обучения.

Варианты архитектур RNN

RNN можно использовать по-разному в зависимости от задачи:

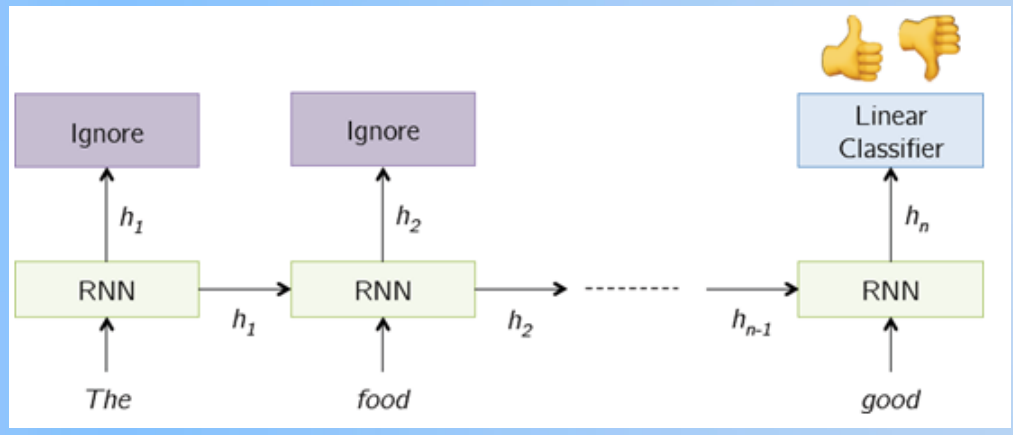
1. **Один ко многим:** описание изображения - на входе одно изображение, на выходе - последовательность слов.
2. **Многие к одному:** определение тональности или темы текста.
3. **Многие ко многим с разной длиной**: машинный перевод, где длина входа и выхода может различаться.
4. **Многие ко многим с одинаковой длиной:** например, классификация каждого кадра видео или разметка каждого слова в предложении (POS-теггинг).



RNN для классификации (два способа)

Допустим, нам нужно получить выход фиксированной длины, например, предсказать тональность отзыва.

Первый способ: использовать только последнее скрытое состояние h_n и подать его в классификатор.



Второй способ: взять все скрытые состояния, например, усреднить их или использовать механизм внимания и затем подать в классификатор.

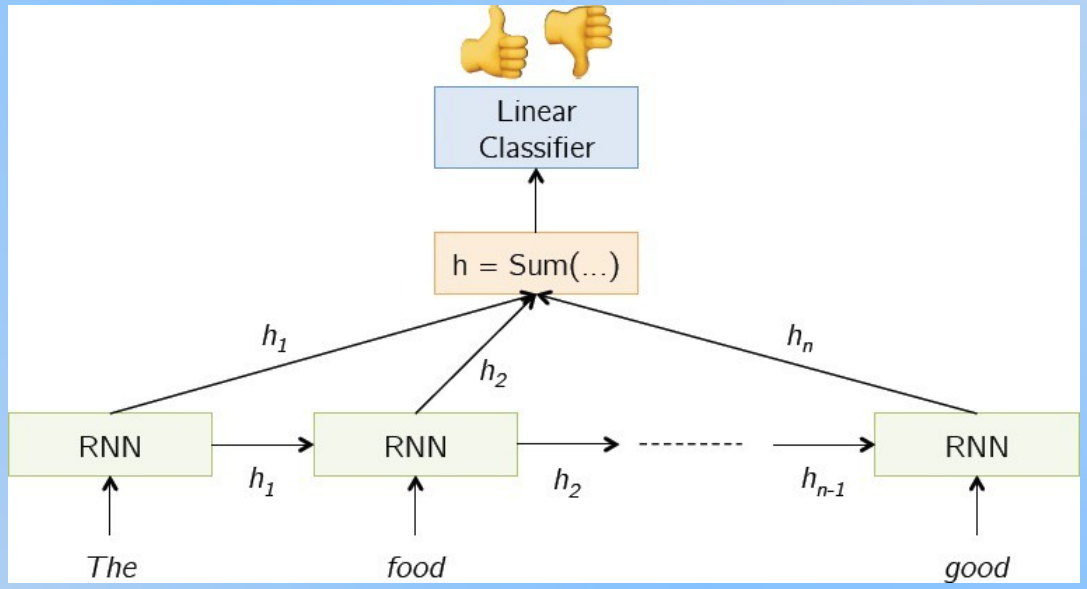
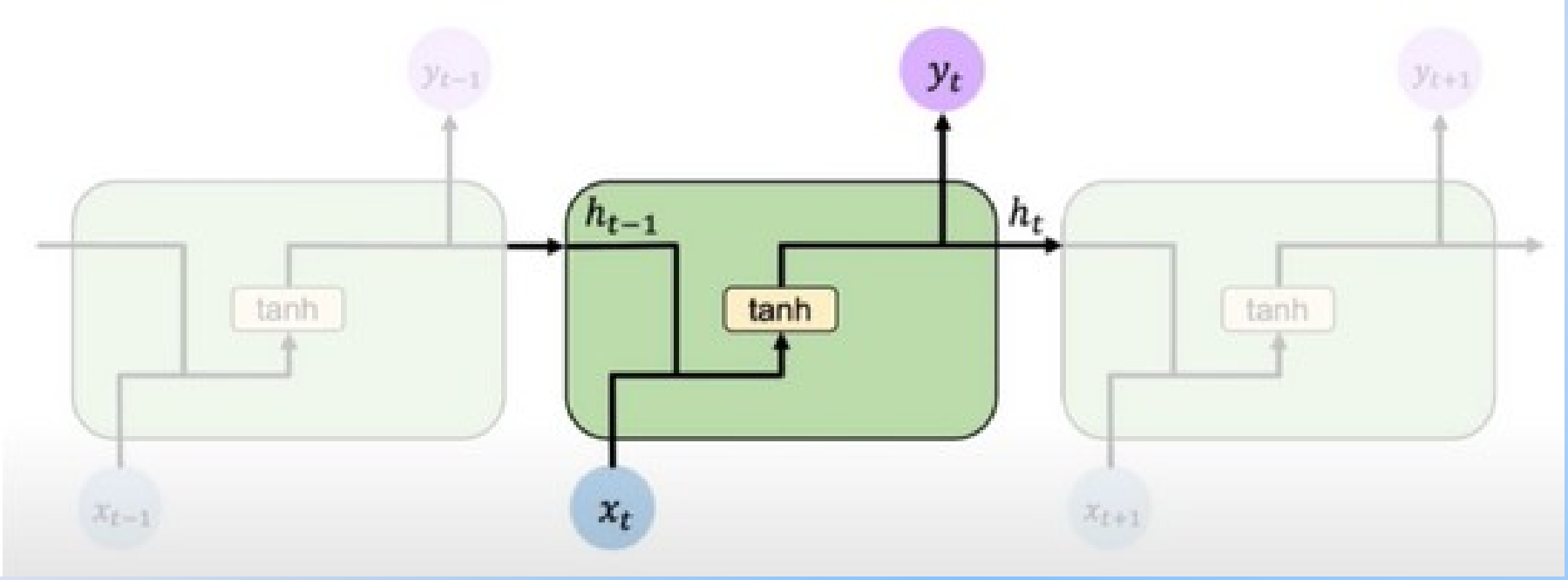


Схема стандартной RNN

В стандартной рекуррентной нейронной сети (Vanilla RNN) повторяющиеся модули содержат простой вычислительный узел, обычно с гиперболическим тангенсом.

В Vanilla RNN есть только h_t, который на каждом шаге полностью пересчитывается.

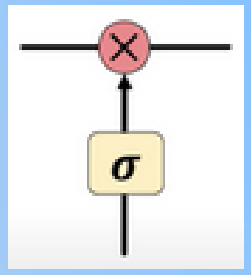


LSTM (1997) - 1

Итак, классическая RNN плохо запоминает долгосрочные зависимости. В 1997 году Зеппом Хохрейтером и Юргеном Шмидхубером была предложена сеть **LSTM (Long Short Term Memory, долгой краткосрочной памяти).**

LSTM вводит **два состояния**: **долговременная память** (cell state) и **выход ячейки** (hidden state) - скрытое состояние, которое мы показываем наружу.

И **три специальных вентиля или гейта**: **забывание** (forget gate), **вход** (input gate) и **выход** (output gate). Эти гейты управляют потоком информации, позволяя сети решать, что забыть, что запомнить и что выдать. Гейты, при необходимости, пропускают информацию, например, через **слой сигмоидной нейронной сети и поточечное умножение**:

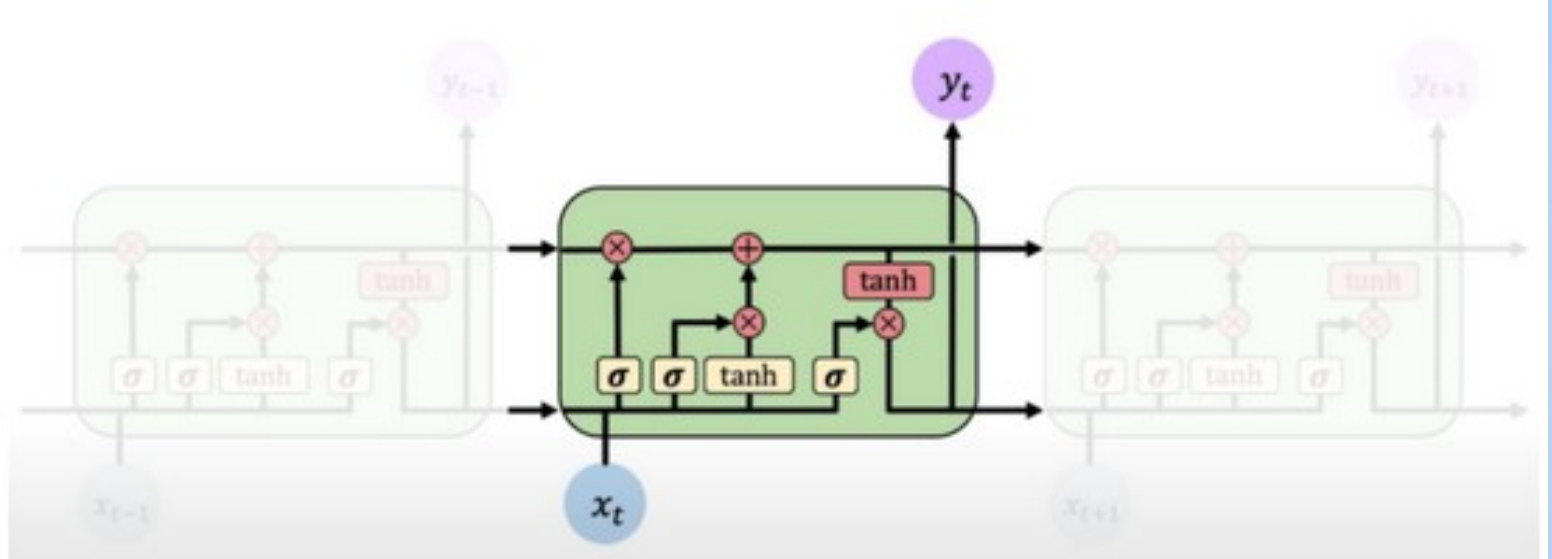


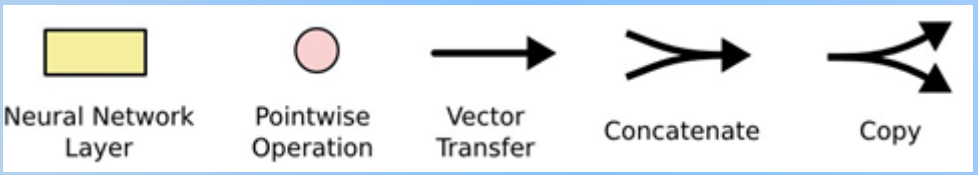
Ячейки LSTM способны отслеживать информацию на протяжении множества временных шагов.

S. Hochreiter, J. Schmidhuber «Long short-term memory» // Neural Computation - 1997. - V. 9, № 8, P. 1735-1780.

LSTM (1997) - 2

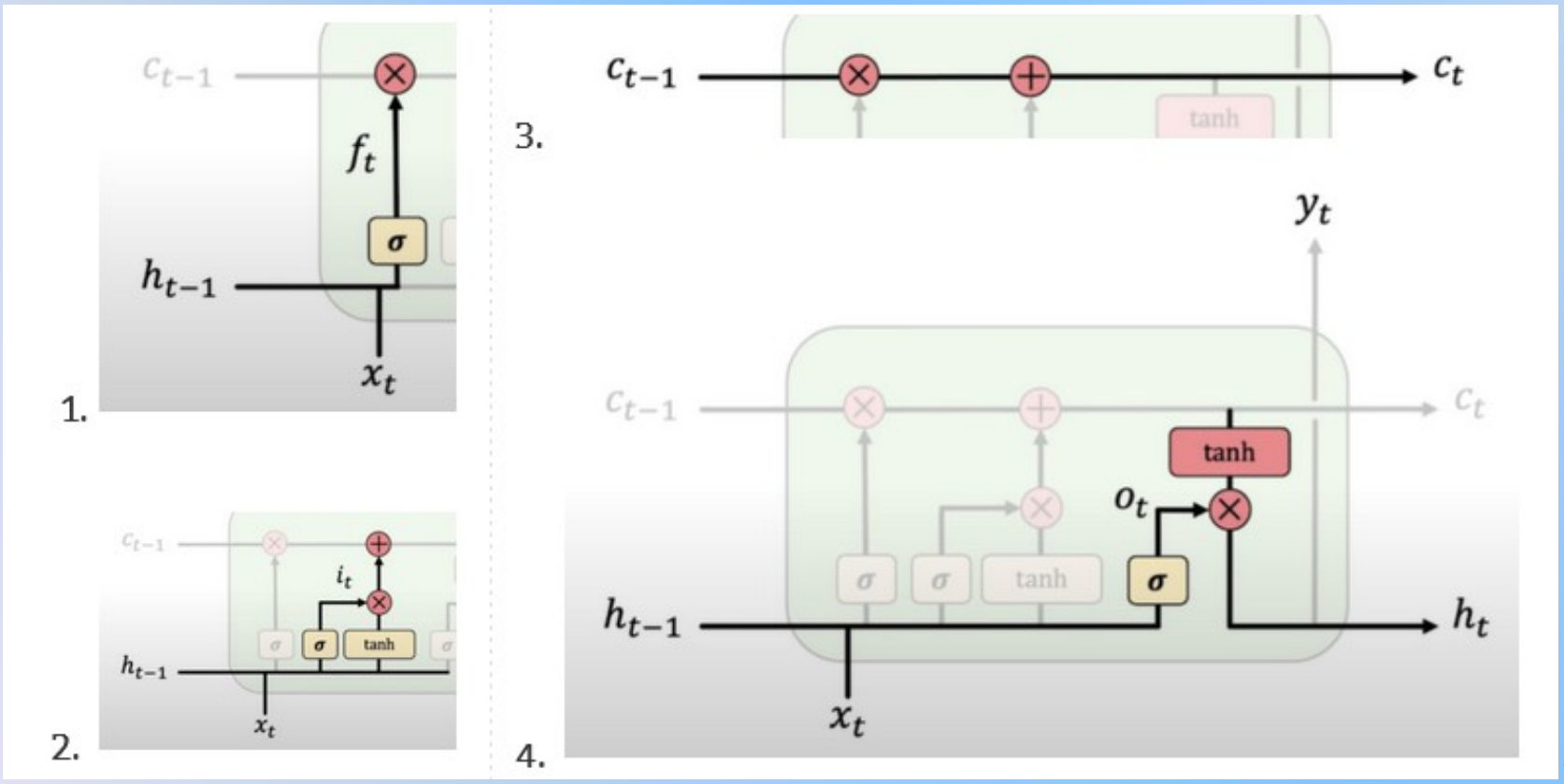
Структура LSTM:





Работа LSTM

1. **Забывание (forget gate).** LSTM-сети забывают нерелевантные части предыдущего состояния.
2. **Сохранение (input gate).** LSTM-сети сохраняют релевантную новую информацию в состоянии ячейки.
3. **Обновление.** LSTM-сети выборочно обновляют значения состояния ячейки.
4. **Вывод (output gate).** Выходной вентиль управляет тем, какая информация отправляется на следующий временной шаг.



Главная инновация LSTM: ячейка памяти (Cell State)

На схеме LSTM сверху, горизонтально, проходит **толстая линия** - это **ячейка памяти** C_t, вектор такой же размерности, как и скрытое состояние h_t, но он играет другую роль. **Ячейка памяти - долговременная память сети,** она проходит через всю последовательность, подвергаясь лишь **небольшим линейным изменениям**.

Формула обновления ячейки памяти:

$$C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t$$

где $\odot$ — поэлементное умножение (произведение Адамара) .

Это **линейная комбинация** старой ячейки C_{t-1} и нового кандидата $\tilde{C}_t$. Коэффициенты-гейты f_t и i_t находятся в диапазоне [0,1].

Это **решает проблему градиента**, так как при обратном распространении градиент через ячейку памяти:

$$\frac{\partial C_t}{\partial C_{t-1}} = f_t$$

Это диагональная матрица с элементами f_t. Если f_t близко к 1, градиент проходит почти без изменений. Нет умножения на матрицу W_{hh} на каждом шаге. Именно поэтому LSTM может запоминать информацию на сотни шагов.

Три гейта - три регулятора - 1

Теперь разберём каждый гейт подробно.

Гейт 1: Forget Gate (гейт забывания) – f_t:

$$f_t = \sigma(W_f[h_{t-1}; x_t] + b_f)$$

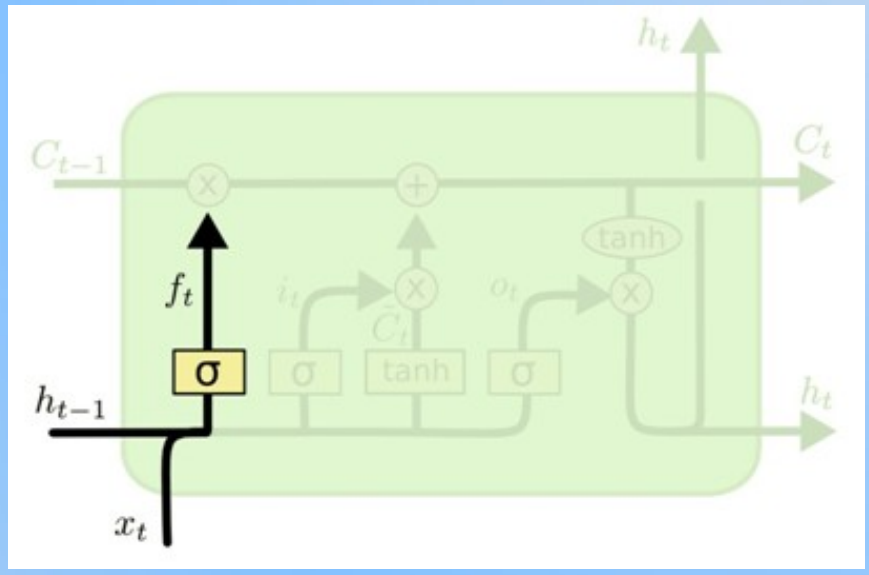
.

Решает, какую информацию из старой ячейки C_{t-1} забыть. **Диапазон:** от 0 (полностью забыть) до 1 (полностью сохранить). В хороших практиках инициализируют b_f близким к 1, чтобы сеть сначала **запоминала**, а потом **училась забывать**.

Сигмоидная функция активации выбрана потому, что её выход лежит в диапазоне [0,1], что идеально подходит для роли «вентиля»:

- значение **1** означает «полностью сохранить эту информацию»;
- значение **0** означает «полностью удалить эту информацию».

Промежуточные значения соответствуют частичному сохранению.



Пример работы гейта забывания

Предположим, мы обрабатываем предложение: «*Мария* живёт в Париже. *Она* работает дизайнером.»

Когда сеть доходит до слова «Она», вентиль забывания может решить, что информация о том, что «Мария» - женщина, важна для правильного согласования, и сохранит её ($f_t \approx 1$).

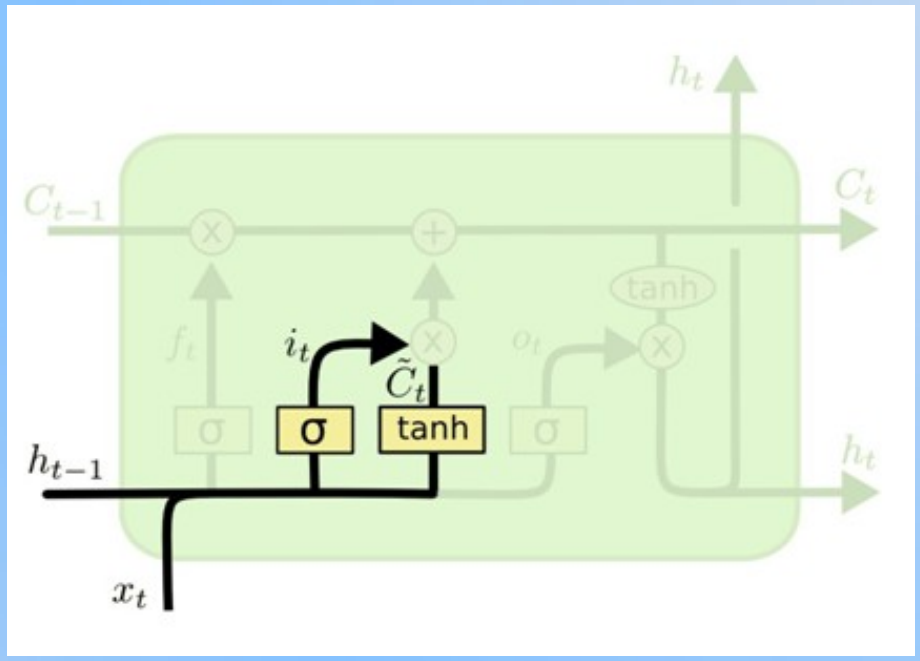
А информация о том, что Мария «живёт», уже не так важна для текущего момента, и её можно частично забыть ($f_t \approx 0.3$).

Три гейта - три регулятора - 2

Гейт 2: Input Gate (входной гейт) – i_t: $i_t = \sigma(W_i[h_{t-1}; x_t] + b_i)$.
Решает, насколько новая информация $C\tilde{}_t$ важна. **Диапазон:** от 0 (игнорировать новое) до 1 (полностью записать).
Параллельно вычисляется **кандидат на новое состояние**: $\tilde{C}_t = \tanh(W_C[h_{t-1}; x_t] + b_C)$.
Это то, что сеть **хотела бы запомнить** из текущего шага.

Для вектора-кандидата используется **гиперболический тангенс**, потому что:

- диапазон [−1,1] позволяет добавлять как положительные, так и отрицательные значения;
- центрированность вокруг нуля помогает избежать смещения;
- нелинейность позволяет моделировать сложные зависимости.



Пример работы входного гейта

Продолжим пример: «Мария живёт в Париже. *Она* работает дизайнером.»

На слове «работает» входной вентиль может решить, что профессия «дизайнер» - это важная новая информация, которую стоит добавить в память.

Вентиль i_t откроется (значения близки к 1) для тех элементов памяти, которые отвечают за профессию, а кандидат $C\tilde{}_t$ предложит значения, кодирующие информацию о дизайнере.

Обновление состояния (Cell update)

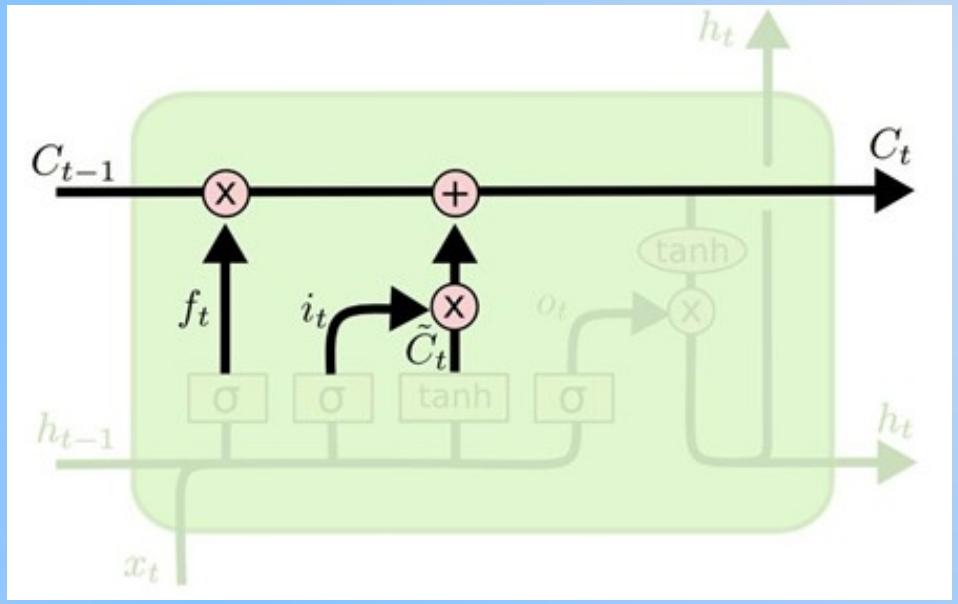
Новое состояние C_t - это взвешенная сумма старого состояния (через forget gate) и кандидата (через input gate):

$$C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t$$

Интерпретация слагаемых:

$f_t \odot C_{t-1}$: старая информация, которую мы решили сохранить. Вентиль забывания f_t поэлементно умножается на старое состояние, «затирая» ненужное.

$i_t \odot \tilde{C}_t$: новая информация, которую мы решили добавить. Входной вентиль i_t регулирует, насколько сильно каждое новое значение из $\tilde{C}_t$ повлияет на память.



Сравнение с простой RNN:

- RNN: $h_t = \tanh(W_h x_t + U_h h_{t-1})$ — градиент умножается на матрицу весов и производную tanh, что ведет к затуханию.
- LSTM: $C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t$ — градиент умножается только на f_t, который сеть может обучить быть близким к 1.

Три гейта - три регулятора - 3

Гейт 3: Output Gate (выходной гейт) – o_t: $o_t = \sigma(W_o[h_{t-1}; x_t] + b_o)$.

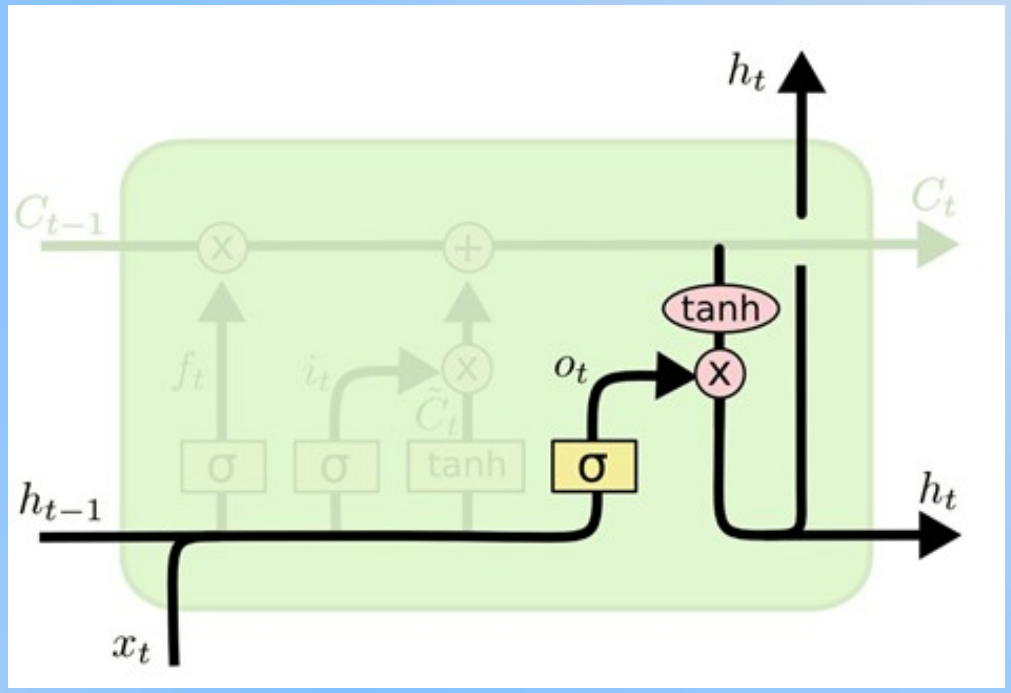
Решает, какую часть ячейки памяти показать наружу в h_t. **Диапазон:** от 0 (ничего не показывать) до 1 (показать всё).

После этого вычисляется **скрытое состояние**: $h_t = o_t \odot \tanh(C_t)$.

Зачем нужен tanh для C_t?

Клеточное состояние C_t может накапливать значения, потенциально выходящие за пределы [−1,1] (из-за многократного сложения).

Пропускание через tanh нормализует значения в диапазон [−1,1], что стабилизирует обучение, предотвращает взрыв активаций, делает выходы более интерпретируемыми.



Пример работы выходного гейта

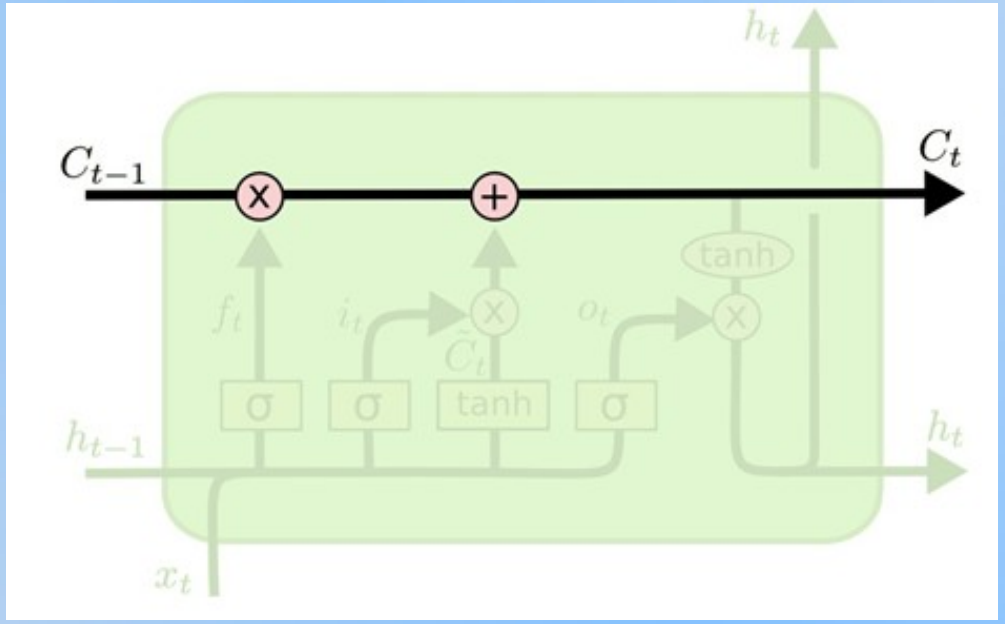
Вернёмся к примеру: «Мария живёт в Париже. Она работает дизайнером.»

Когда сеть генерирует слово «дизайнером», выходной гейт должен извлечь из памяти информацию о профессии, добавленную на слове «работает», и подавить нерелевантную информацию (например, о месте жительства).

Полный цикл LSTM

На каждом шаге *t*:

1. **Забываем** часть старой памяти: $f_t \odot C_{t-1}$
2. **Запоминаем** новую информацию: $i_t \odot \tilde{C}_t$
3. **Обновляем** ячейку: $C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t$
4. **Решаем, что показать:** $o_t = \sigma(W_o[h_{t-1}; x_t] + b_o)$
5. **Вычисляем** скрытое состояние: $h_t = o_t \odot \tanh(C_t)$



Размерности и количество параметров LSTM

Размерности:

- Вход: $x_t \in \mathbb{R}^{d_x}$
- Состояния: $h_t, C_t \in \mathbb{R}^{d_h}$
- Конкатенация: $[h_{t-1}, x_t] \in \mathbb{R}^{d_h+d_x}$
- Весовые матрицы: $W_f, W_i, W_C, W_o \in \mathbb{R}^{d_h \times (d_h+d_x)}$
- Смещения: $b_f, b_i, b_C, b_o \in \mathbb{R}^{d_h}$

Общее **количество обучаемых параметров** в одном LSTM-слое:

$$N_{params} = 4 \cdot d_h \cdot (d_h + d_x + 1)$$

Пример: Для $d_x = 100$, $d_h = 256$:

$$N_{params} = 4 \cdot 256 \cdot (256 + 100 + 1) = 4 \cdot 256 \cdot 357 = 365,568$$

Сравнение с простой RNN: $d_h \cdot (d_h + d_x + 1) = 256 \cdot 357 = 91,392$. LSTM имеет в 4 раза больше параметров.

Карусель константной ошибки в LSTM

Карусель константной ошибки (Constant Error Carousel, CEC) - ключевой механизм в архитектуре LSTM (Long Short-Term Memory), позволяющий решить **проблему исчезающего** градиента (затухания градиента) в рекуррентных нейронных сетях.

Рассмотрим, как градиент функции потерь по клеточному состоянию C_t распространяется назад к C_{t-1}. Из уравнения обновления $C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t$:

$$\frac{\partial c_t}{\partial c_{t-1}} = \mathrm{diag}(f_t)$$

В LSTM производная зависит **только** от значений вентиля забывания f_t. В ней нет умножения на весовую матрицу и производную нелинейной функции активации tanh, поэтому:

- Сеть может **научиться** держать $f_t \approx 1$ для элементов памяти, которые важны в долгосрочной перспективе.
- При $f_t \approx 1$, $\frac{\partial C_t}{\partial C_{t-1}} \approx I$ (единичная матрица).
- Градиент проходит через временные шаги **без затухания**.

Инициализация смещения вентиля забывания: именно поэтому рекомендуется инициализировать b_f положительным значением (обычно 1.0 или 2.0), чтобы вентиль изначально был **открыт**, это запускает механизм CEC с первых шагов обучения, что даёт сети «презумпцию сохранения» информации. Модель учится **активно закрывать** вентиль (f_t->0), чтобы удалить данные, а не наоборот.

Недостатки LSTM

LSTM - мощная архитектура, но у неё есть и **минусы**:

- **Много параметров** - 8 матриц (против 2-3 в Vanilla RNN). Больше данных для обучения.
- **Медленнее обучается** из-за большего числа операций на шаг.
- **Сложнее интерпретировать:** что именно делает каждый гейт, не всегда очевидно.
- **Всё ещё может затухать:** если forget gate систематически маленький, градиент затухнет.

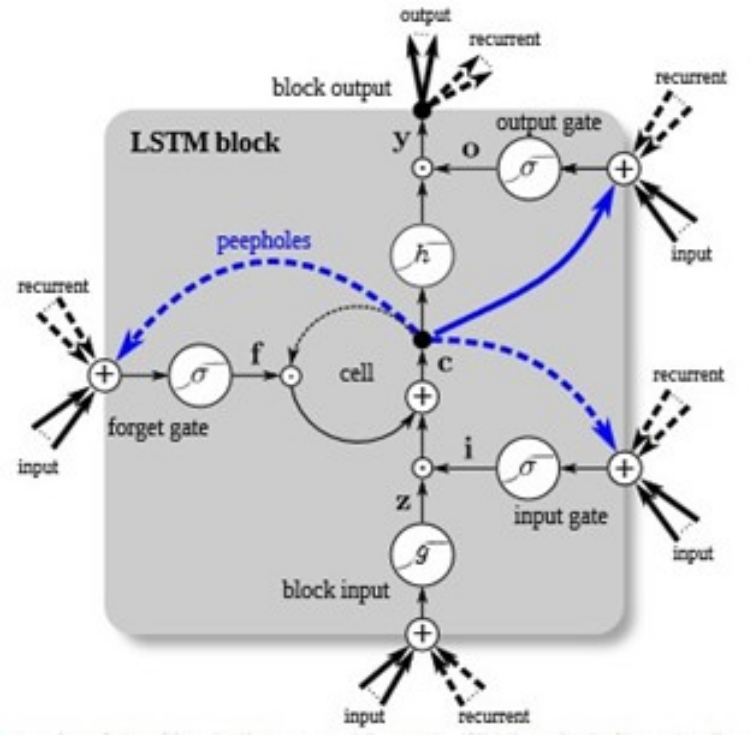
Именно поэтому позже появился GRU - упрощённая версия LSTM с 2 гейтами.

LSTM с peephole connections (связями-«глазками») - 1

В классической LSTM каждый гейт вычисляется на основе **предыдущего скрытого состояния** h_{t-1} и **текущего входа** x_t.

Феликс Герс, Юрген Шмидхубер, Фред Камминс в 2000 году предложили модификацию LSTM с **peephole connections (связями-«глазками»)**, когда гейты зависят не только от предыдущего скрытого состояния (hidden state) h_{t-1}, но и от предыдущего состояния ячейки (cell state) C_{t-1}.

Peephole в переводе с английского - «глазок» (как дверной глазок). В контексте LSTM это дополнительные связи, которые позволяют гейтам **подглядывать** в ячейку памяти.



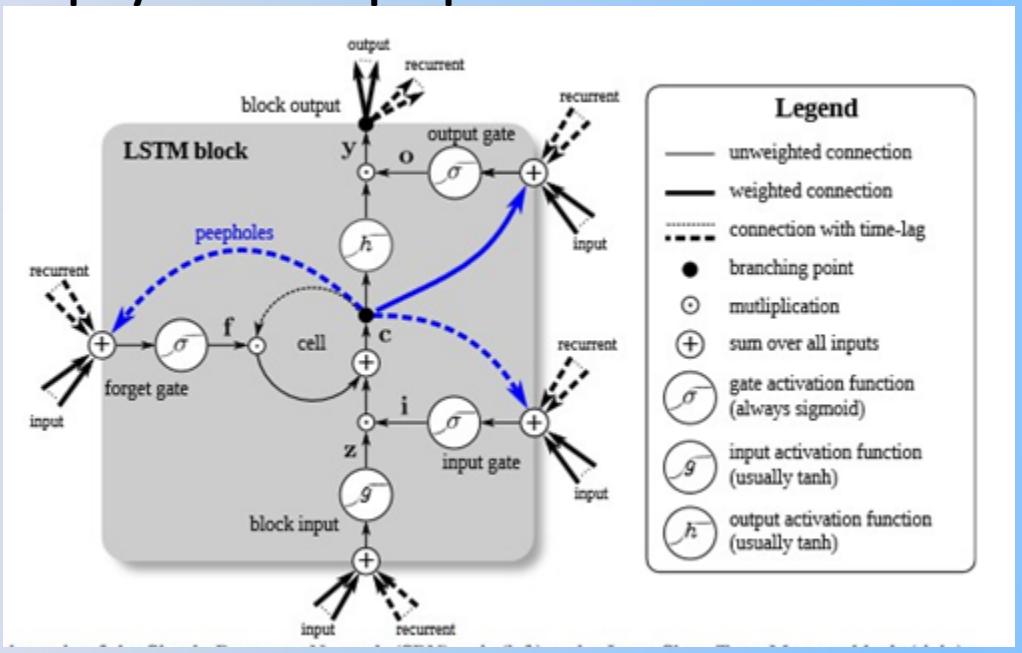
На схеме это изображается дополнительными стрелками:

- От C_{t-1} к forget gate f_t
- От C_{t-1} к input gate i_t
- От C_t (новое состояние) к output gate o_t

Когда гейт решает, забывать ли старую память, ему полезно знать, что именно хранится в этой памяти. А когда решает, что выводить, полезно знать новое состояние ячейки.

LSTM с peephole connections (связями-«глазками») - 2

Формулы LSTM с peephole:



$$f_t = \sigma(W_f[h_{t-1}; x_t; C_{t-1}] + b_f)$$
$$i_t = \sigma(W_i[h_{t-1}; x_t; C_{t-1}] + b_i)$$
$$o_t = \sigma(W_o[h_{t-1}, x_t; C_t] + b_o)$$
$$\tilde{C}_t = \tanh(W_C[h_{t-1}; x_t] + b_C)$$
$$C_t = f_t C_{t-1} + i_t \tilde{C}_t$$
$$h_t = o_t \tanh(C_t)$$

Гейт забывания и входной вентиль получают доступ к **предыдущему** клеточному состоянию C_{t-1}. Выходной гейт получает доступ к **текущему** клеточному состоянию C_t.

Преимущества:

- вентили могут принимать более информированные решения на основе актуального содержимого памяти;
- особенно полезно для задач, требующих точного тайминга (например, распознавание речи с паузами);
- только небольшое увеличение числа параметров (добавляются диагональные матрицы для peephole-соединений).

Сравнение классической LSTM и LSTM с peephole

Исследователи сравнивали классическую LSTM и LSTM с peephole на разных задачах:

Задача	Классическая **LSTM**	**LSTM** с **peephole**	Улучшение
Музыкальная генерация	базовая	небольшое улучшение	±
Распознавание речи	базовая	иногда лучше	иногда
Моделирование языка	базовая	незначительно лучше	редко
TIMIT (фонемы)	базовая	сопоставима	нет

Peephole connections **иногда** дают небольшое улучшение, но не являются панацеей. В большинстве современных реализаций LSTM (например, в PyTorch и TensorFlow) peephole **не используются по умолчанию**, но могут быть включены опционально.

Алгоритм BPTT для LSTM

Алгоритм обратного распространения во времени (Backpropagation Through Time, BPTT) состоит из прямого и обратного проходов.

Прямой проход (Forward Pass):

1. Инициализировать $h_0 = \mathbf{0}$, $C_0 = \mathbf{0}$
2. Для $t = 1, 2, ..., T$:
   - Вычислить $f_t, i_t, \tilde{C}_t, o_t$ по формулам LSTM
   - Обновить $C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t$
   - Вычислить $h_t = o_t \odot \tanh(C_t)$
   - Вычислить выход $y_t = W_y h_t + b_y$ (если требуется)
   - **Сохранить** все промежуточные значения для обратного прохода

   Требования к памяти: $O(T \cdot d_h)$ для хранения всех $h_t, C_t, f_t, i_t, \tilde{C}_t, o_t$.

Обратный проход (Backward Pass):

1. Инициализировать $\frac{\partial L}{\partial h_{T+1}} = \mathbf{0}$, $\frac{\partial L}{\partial C_{T+1}} = \mathbf{0}$
2. Для $t = T, T-1, ..., 1$:
   - Получить $\frac{\partial L_t}{\partial y_t}$ из функции потерь
   - Вычислить $\frac{\partial L}{\partial h_t} = \frac{\partial L_t}{\partial h_t} + \frac{\partial L}{\partial h_{t+1}} \frac{\partial h_{t+1}}{\partial h_t}$
   - Вычислить $\frac{\partial L}{\partial C_t}$ с учетом вклада от h_t и C_{t+1}
   - Вычислить градиенты по вентилям $\delta o_t, \delta f_t, \delta i_t, \delta \tilde{C}_t$
   - Накопить градиенты по весам W_o, W_f, W_i, W_C

Функция потерь:

$$L = \sum_{t=1}^{T} L_t(y_t, y_t^{true})$$

где L_t - потеря на шаге t (например, кросс-энтропия для классификации).

Ключевое рекуррентное соотношение для LSTM

Клеточное состояние C_t влияет на общую функцию потерь L двумя путями:

1. **Напрямую через выход текущего шага:** $C_t \to h_t \to L_t$
2. **Косвенно через состояние следующего шага:** $C_t \to C_{t+1} \to L_{t+1}, ..., L_T$

Градиент по клеточному состоянию, то есть полная производная по C_t является суммой двух вкладов:

$$\delta C_t = \frac{\partial L}{\partial C_t} = \underbrace{\frac{\partial L}{\partial h_t}\frac{\partial h_t}{\partial C_t}}_{\text{Вклад от } h_t} + \underbrace{\frac{\partial L}{\partial C_{t+1}}\frac{\partial C_{t+1}}{\partial C_t}}_{\text{Вклад от } C_{t+1}}$$

Первое слагаемое:

$$h_t = o_t \odot \tanh(C_t)$$

$$\frac{\partial h_t}{\partial C_t} = \text{diag}(o_t \odot (1 - \tanh^2(C_t)))$$

$$\frac{\partial L}{\partial h_t}\frac{\partial h_t}{\partial C_t} \equiv \frac{\partial L}{\partial h_t} \odot o_t \odot (1 - \tanh^2(C_t))$$

Второе слагаемое:

$$C_{t+1} = f_{t+1} \odot C_t + i_{t+1} \odot \tilde{C}_{t+1}$$

$$\frac{\partial C_{t+1}}{\partial C_t} = \text{diag}(f_{t+1})$$

$$\frac{\partial L}{\partial C_{t+1}}\frac{\partial C_{t+1}}{\partial C_t} \equiv \delta C_{t+1} \odot f_{t+1}$$

Итоговая формула для **градиента клеточного состояния**:

$$\delta C_t = \frac{\partial L}{\partial h_t} \odot o_t \odot \left(1 - \tanh^2(C_t)\right) + \delta C_{t+1} \odot f_{t+1}$$

Первое слагаемое - вклад от текущего выхода h_t.

Второе слагаемое - вклад от будущих шагов через умножение на f_{t+1}, карусель константной ошибки (CEC) в действии.

Другие градиенты

Градиенты по параметрам вентилей:

$$\delta o_t = \frac{\partial L}{\partial h_t} \odot \tanh(C_t) \odot \sigma'(o_t^{input})$$
$$\delta f_t = \delta C_t \odot C_{t-1} \odot \sigma'(f_t^{input})$$
$$\delta i_t = \delta C_t \odot \tilde{C}_t \odot \sigma'(i_t^{input})$$
$$\delta \tilde{C}_t = \delta C_t \odot i_t \odot \left(1 - \tanh^2(\tilde{C}_t^{input})\right)$$

где $*^{input}$ обозначает аргумент соответствующей функции активации до ее применения.

Накопление градиентов по весам:

$$\frac{\partial L}{\partial W_f} = \sum_{t=1}^{T} \delta f_t \cdot [h_{t-1}, x_t]^T$$
$$\frac{\partial L}{\partial W_i} = \sum_{t=1}^{T} \delta i_t \cdot [h_{t-1}, x_t]^T$$
$$\frac{\partial L}{\partial W_C} = \sum_{t=1}^{T} \delta \tilde{C}_t \cdot [h_{t-1}, x_t]^T$$
$$\frac{\partial L}{\partial W_o} = \sum_{t=1}^{T} \delta o_t \cdot [h_{t-1}, x_t]^T$$

Накопление градиентов по смещениям:

$$\frac{\partial L}{\partial b_f} = \sum_{t=1}^{T} \delta f_t$$
$$\frac{\partial L}{\partial b_i} = \sum_{t=1}^{T} \delta i_t$$
$$\frac{\partial L}{\partial b_C} = \sum_{t=1}^{T} \delta \tilde{C}_t$$
$$\frac{\partial L}{\partial b_o} = \sum_{t=1}^{T} \delta o_t$$

TBPTT - обучение на длинных последовательностях

Для очень длинных последовательностей (например, аудиофайл длительностью 1 час) классический BPTT сталкивается с **проблемами**:

- высокие требования к памяти - нужно хранить все состояния сети и градиенты для каждого временного шага;
- долгие вычисления - каждая итерация обучения проходит через все *T* шагов;
- градиенты - распространение через сотни тысяч шагов может быть численно нестабильным.

Алгоритм усечённого обратного распространения во времени (Truncated BPTT, TBPTT), предложенный Рональдом Вильямсом, Цзином Пэном (Ronald J. Williams, Jing Peng) в 1990 г. решает эти проблемы, разбивая последовательность на сегменты и ограничивая обратное распространение фиксированным количеством шагов *к* (горизонт или длина усечения):

1. **Разбиение**. Длинная последовательность $x_1, x_2, \ldots, x_T$ делится на блоки:

$$[x_1, \ldots, x_k], \quad [x_{k+1}, \ldots, x_{2k}], \quad \ldots$$

2. **Прямой проход**. Для каждого блока вычисляются скрытые состояния и потери.
3. **Обратный проход**. Градиенты рассчитываются **только внутри текущего блока** (длиной k шагов), а не по всей последовательности.
4. **Обновление весов**. Параметры сети корректируются на основе полученных градиентов.
5. **Переход к следующему блоку**. Скрытое состояние с конца текущего блока передаётся как начальное состояние для следующего.

Выбор горизонта усечения

- *к* = 1: эквивалентно RTRL (рассмотрим ниже), нет долгосрочных зависимостей;
- *к* = 10–20: улавливает локальные зависимости, быстрое обучение;
- *к* = 50–100: хороший баланс для многих задач обработки естественного языка (NLP);
- *к* = *T*: классический BPTT, максимальное качество, медленно.

Для языковых моделей обычно используют *к* = 35–50.

Williams R. J., Peng J. (1990) "An efficient gradient-based algorithm for on-line training of recurrent network trajectories" // Neural Computation, 2(4), 490-501.

RTRL - онлайн-обучение - 1

Алгоритм **рекуррентного обучения в реальном времени (Real-Time Recurrent Learning, RTRL) -** это алгоритм, предложенный в 1989 г. Рональдом Вильямсом и Дэвидом Ципсером (Ronald J. Williams, David Zipser), который вычисляет точные градиенты функции потерь по параметрам сети **в реальном времени**, по мере поступления входных данных, без необходимости «разворачивания» сети во времени.

Ключевая идея - вместо того чтобы хранить всю историю состояний и затем распространять ошибку назад (как в BPTT), RTRL «несёт» градиентную информацию вперёд вместе с состоянием сети.

В алгоритме RTRL ключевую роль играет **матрица чувствительности** (sensitivity matrix), которая отслеживает, как изменение каждого параметра сети влияет на текущее скрытое состояние h_t. В отличие от BPTT, где градиенты вычисляются после обработки всей последовательности, RTRL обновляет эту матрицу рекуррентно на каждом временном шаге, что теоретически позволяет обучать сеть онлайн.

RTRL - онлайн-обучение - 2

Рассмотрим классическую рекуррентную сеть (RNN):

$$h_t = \sigma\left(W_{hx}x_t + W_{hh}h_{t-1} + b_h\right)$$

$$y_t = W_{yh}h_t + b_y$$

Обозначим через θ любой обучаемый параметр сети (элемент матриц W_{hx}, W_{hh}, W_{yh} или векторов b_h, b_y). Матрица чувствительности для скрытого состояния определяется как производная $\partial h_t/\partial\theta$.

Применяя правило цепочки к уравнению скрытого состояния, получаем **рекуррентную формулу обновления матрицы чувствительности**:

$$\frac{\partial h_t}{\partial \theta} = \sigma'(\cdot) \odot \left(\frac{\partial}{\partial \theta}\left(W_{hx}x_t + W_{hh}h_{t-1} + b_h\right) + W_{hh}\frac{\partial h_{t-1}}{\partial \theta} \right)$$

После вычисления $\partial h_t/\partial\theta$ градиент функции потерь L_t по параметру θ вычисляется через выходной слой:

$$\frac{\partial L_t}{\partial \theta} = \frac{\partial L_t}{\partial y_t} \cdot W_{yh} \cdot \frac{\partial h_t}{\partial \theta}$$

Это позволяет обновить веса немедленно, не дожидаясь окончания последовательности.

RTRL - онлайн-обучение - 3

Пояснения к формуле:

$$\frac{\partial h_t}{\partial \theta} = \sigma'(\cdot) \odot \left(\frac{\partial}{\partial \theta} \left(W_{hx} x_t + W_{hh} h_{t-1} + b_h \right) + W_{hh} \frac{\partial h_{t-1}}{\partial \theta} \right)$$

1. $\sigma'(\cdot)$ — производная функции активации, вычисленная в точке $W_{hx} x_t + W_{hh} h_{t-1} + b_h$. Это вектор, который поэлементно умножается (произведение Адамара $\odot$) на выражение в скобках.
2. **Первое слагаемое в скобках** $\frac{\partial}{\partial \theta} \left(W_{hx} x_t + W_{hh} h_{t-1} + b_h \right)$ — это **прямой вклад** параметра θ в текущее скрытое состояние. Его вид зависит от того, где находится θ:
   - Если $\theta = W_{hx}^{ij}$ (элемент матрицы W_{hx}), то производная равна вектору, у которого i-я компонента равна x_t^j, а остальные нули.
   - Если $\theta = W_{hh}^{ij}$ (элемент матрицы W_{hh}), то производная равна вектору, у которого i-я компонента равна h_{t-1}^j, а остальные нули.
   - Если $\theta = b_h^i$ (элемент смещения b_h), то производная равна единичному вектору с единицей на i-й позиции.
3. **Второе слагаемое** $W_{hh} \frac{\partial h_{t-1}}{\partial \theta}$ — это **рекуррентный вклад**. Оно показывает, как изменение параметра θ повлияло на предыдущее скрытое состояние h_{t-1} (что уже вычислено и хранится в матрице чувствительности с прошлого шага) и как это влияние передаётся в текущий момент через умножение на рекуррентную матрицу W_{hh}. Именно это слагаемое делает формулу рекуррентной: чтобы вычислить $\frac{\partial h_t}{\partial \theta}$, нам необходимо знать $\frac{\partial h_{t-1}}{\partial \theta}$.

Применение RTRL к LSTM

Хотя формула выше выписана для простой RNN, её логика полностью переносится и на LSTM, однако с экспоненциальным ростом сложности.

В LSTM вместо одного уравнения состояния h_t есть система из шести уравнений. Для каждого состояния (например, клеточного состояния C_t) и параметра θ (формула будет иметь вид:

$$\frac{\partial C_t}{\partial \theta} = \frac{\partial}{\partial \theta}\left(f_t \odot C_{t-1} + i_t \odot \tilde{C}_t\right)$$

При раскрытии этой производной по правилу произведения и цепочки появятся:

- $\frac{\partial f_t}{\partial \theta}$, которая будет включать $\frac{\partial h_{t-1}}{\partial \theta}$ и рекуррентную матрицу W_{fh},
- $\frac{\partial i_t}{\partial \theta}$ с матрицей W_{ih},
- $\frac{\partial \tilde{C}_t}{\partial \theta}$ с матрицей W_{Ch},
- $\frac{\partial C_{t-1}}{\partial \theta}$ (рекуррентная зависимость от предыдущего клеточного состояния).

Каждая из этих производных будет иметь свою рекуррентную формулу, аналогичную приведённой выше для простой RNN, но с соответствующими матрицами.

Проблемы применения RTRL к LSTM

1. Вычислительная сложность: в LSTM количество параметров примерно в 4 раза больше, чем в простой RNN с той же размерностью скрытого состояния. RTRL требует хранения и обновления матрицы чувствительности размером $n \times |\theta|$, где n - количество нейронов, а $|\theta|$ - общее число параметров.

Для LSTM это приводит к сложности $O(n^4)$ по времени и $O(n^3)$ по памяти, что делает его абсолютно непрактичным для сетей размером более нескольких десятков нейронов.

2. Экспоненциальное затухание ошибки: даже в LSTM, несмотря на механизм карусели постоянной ошибки (CEC) для C_t, производные по параметрам *гейтов* (W_{fh}, W_{ih}, W_{oh}) всё равно проходят через нелинейности (сигмоиды и тангенсы) и умножения на веса.

При рекуррентном вычислении в RTRL эти производные могут экспоненциально затухать, что сводит на нет способность LSTM к долгосрочной памяти в контексте этого конкретного алгоритма обучения.

Современное состояние и гибридные подходы к обучению LSTM

Из-за указанных проблем **чистый RTRL для классической LSTM практически не используется**. Однако идея онлайн-обучения привлекательна, поэтому развиваются гибридные методы:

- **eLSTM (economical LSTM):** упрощённая архитектура LSTM, для которой можно применять аппроксимации RTRL с меньшей вычислительной сложностью;
- **Truncated Hybrid:** комбинация усечённого BPTT (для точного расчёта локальных градиентов) и усечённого RTRL (для аппроксимации влияния прошлого за пределами окна усечения);
- **NoBackTrack и RFLO:** аппроксимации полной матрицы чувствительности с помощью случайных проекций или стохастических оценок, что радикально снижает сложность и шум.

Таким образом, применение классического RTRL к LSTM является теоретически возможным, но вычислительно нереализуемым для задач реального масштаба. На практике для онлайн-обучения LSTM используются специализированные архитектурные упрощения или гибридные алгоритмы, аппроксимирующие поведение RTRL.

Сравнение сложности алгоритмов обучения LSTM

Алгоритм	Время на шаг	Память
BPTT	$O(n^2)$	$O(Tn)$
TBPTT(k)	$O(kn^2)$	$O(kn)$
RTRL	$O(n^4)$	$O(n^3)$

Здесь *n* - количество нейронов, *T* - длина последовательности, *k* - горизонт усечения.